Java Level 5 — Modern Java

Overview

Level 5 focuses on **Modern Java features**, especially Lambda Expressions, Functional Interfaces, Streams, Collectors, and Optional.

The goal is not to memorize every Stream API method. The goal is to understand the common patterns used in real Java backend applications.

---

1. Lambda Expressions

A lambda is a concise way to represent behavior.

Syntax

```
(parameters) -> expression
```

or:

```
(parameters) -> {
    statements;
}
```

Example

```
(a, b) -> a + b
```

```
employee -> employee.name()
```

```
employee -> System.out.println(employee.name())
```

Mental Model

```
Lambda
   ↓
```

```
Pass behavior as a value
```

---

2. Functional Interfaces

A functional interface contains exactly **one abstract method**.

```
@FunctionalInterface
interface Calculator {

    int calculate(int a, int b);
}
```

Using a lambda:

```
Calculator addition =
        (a, b) -> a + b;

Calculator multiplication =
        (a, b) -> a * b;
```

```
addition.calculate(10, 20);
```

Mental Model

```
Functional Interface
        ↓
Defines the shape of behavior

Lambda
        ↓
Provides the implementation
```

---

3. Important Functional Interfaces

The four most important ones are:

Interface	Input	Output	Method
`Predicate<T>`	T	boolean	`test()`

`Function<T,R>`	T	R	`apply()`
`Consumer<T>`	T	void	`accept()`
`Supplier<T>`	none	T	`get()`

Predicate

Used for conditions.

```
Predicate<Employee> isSenior =
        employee -> employee.age() >= 30;
```

```
boolean result =
        isSenior.test(employee);
```

Mental model:

```
T → boolean
```

Function

Transforms one value into another.

```
Function<Employee, String> getName =
        employee -> employee.name();
```

```
String name =
        getName.apply(employee);
```

Mental model:

```
T → R
```

Consumer

Consumes a value and performs an action.

```
Consumer<Employee> printEmployee =
        employee ->
                System.out.println(employee.name());
```

```
printEmployee.accept(employee);
```

Mental model:

```
T → void
```

Supplier

Produces a value without receiving input.

```
Supplier<String> generateId =
        () -> UUID.randomUUID().toString();
```

```
String id =
        generateId.get();
```

Mental model:

```
() → T
```

---

4. Method References

Method references provide a shorter syntax for certain lambdas.

Instead of:

```
employees.stream()
        .map(employee -> employee.name())
        .toList();
```

Use:

```
employees.stream()
        .map(Employee::name)
        .toList();
```

Another example:

```
employees.forEach(
        System.out::println
);
```

Common forms

```
ClassName::staticMethod
```

```
object::instanceMethod
```

```
ClassName::instanceMethod
```

```
ClassName::new
```

Constructor reference:

```
Supplier<Employee> supplier =
        Employee::new;
```

---

5. Stream API

Streams allow collections to be processed declaratively.

Basic pipeline

```
Source
  ↓
Intermediate operations
  ↓
Terminal operation
```

Example:

```
List<String> names =
        employees.stream()
                .filter(e -> e.age() >= 30)
                .map(Employee::name)
                .toList();
```

Flow:

```
employees
    ↓
stream()
    ↓
filter()
    ↓
map()
    ↓
toList()
```

---

6. `filter()`

Filters elements based on a condition.

```
List<Employee> seniors =
        employees.stream()
                .filter(
                    employee -> employee.age() >= 30
                )
                .toList();
```

Mental model:

```
Element
   ↓
Predicate
   ↓
keep / discard
```

---

7. `map()`

Transforms elements.

```
List<String> names =
        employees.stream()
                .map(Employee::name)
                .toList();
```

Transformation:

```
Employee
   ↓
name()
   ↓
String
```

Common pattern

```
employees.stream()
        .filter(e -> e.age() >= 30)
        .map(Employee::name)
        .toList();
```

Read as:

> Get the names of employees whose age is at least 30.

---

8. `flatMap()`

`flatMap()` is used when each element produces multiple values and you want to flatten them.

Suppose:

```
Employee 1 → [Project A, Project B]
Employee 2 → [Project C, Project D]
```

Using `map()`:

```
Stream<List<Project>>
```

Using `flatMap()`:

```
Stream<Project>
```

Example:

```
List<Project> projects =
        employees.stream()
                .flatMap(
                    employee ->
                            employee.projects().stream()
                )
                .toList();
```

Remember

```
map
1 → 1

flatMap
1 → many → flatten
```

This is particularly useful for nested API or database data.

---

9. `reduce()`

`reduce()` combines multiple values into a single value.

```
int total =
        numbers.stream()
                .reduce(
                    0,
                    (a, b) -> a + b
                );
```

Or:

```
int total =
        numbers.stream()
                .reduce(
                    0,
                    Integer::sum
                );
```

Example:

```
1 + 2 + 3 + 4
      ↓
     10
```

Prefer specialized operations when available

Prefer:

```
numbers.stream().mapToInt(x -> x).sum();
```

over:

```
numbers.stream()
        .reduce(0, Integer::sum);
```

Use `reduce()` when the aggregation isn't already provided by a specialized operation.

---

10. `collect()`

`collect()` gathers Stream results.

Example:

```
List<String> names =
        employees.stream()
                .map(Employee::name)
                .collect(Collectors.toList());
```

Modern Java can often use:

```
.toList();
```

But understanding `collect()` is essential because it enables collectors such as:

```
groupingBy()
counting()
summingDouble()
partitioningBy()
```

11. `groupingBy()`

Groups elements based on a property.

```
Map<String, List<Employee>> employeesByDepartment =
        employees.stream()
                .collect(
                    Collectors.groupingBy(
                        Employee::department
                    )
                );
```

Result:

```
Engineering → [Ramu, Alex]
Finance     → [John, Mike]
HR          → [Sam]
```

Mental model:

```
Group by field
      ↓
Map<Key, List<Value>>
```

12. `groupingBy()` + `counting()`

Count elements in each group.

```
Map<String, Long> countByDepartment =
        employees.stream()
                .collect(
                    Collectors.groupingBy(
                        Employee::department,
                        Collectors.counting()
                    )
                );
```

Result:

```
Engineering → 10
Finance     → 5
HR          → 3
```

---

13. `groupingBy()` + Aggregation

Calculate total salary per department.

```
Map<String, Double> salaryByDepartment =
        employees.stream()
                .collect(
                    Collectors.groupingBy(
                        Employee::department,
                        Collectors.summingDouble(
                            Employee::salary
                        )
                    )
                );
```

This pattern is extremely useful for reporting and backend data processing.

---

14. `partitioningBy()`

`partitioningBy()` splits elements into exactly two groups:

```
true
false
```

Example:

```
Map<Boolean, List<Employee>> result =
        employees.stream()
                .collect(
                    Collectors.partitioningBy(
                        employee ->
                                employee.salary() >= 100000
                    )
                );
```

Result:

```
true  → salary >= 100000
false → salary < 100000
```

Difference

```
groupingBy()
→ multiple groups

partitioningBy()
→ exactly two groups
```

---

15. Optional

`Optional<T>` represents:

```
value exists
OR
value doesn't exist
```

Example:

```
Optional<Employee> employee =
        employees.stream()
                .filter(e -> e.id() == 10)
                .findFirst();
```

Instead of returning `null`, a method can return:

```
Optional<Employee>
```

---

16. `orElse()`

Provides a default value.

```
Employee employee =
        employeeOptional.orElse(
                defaultEmployee
        );
```

If the value exists:

```
return existing value
```

Otherwise:

```
return default value
```

---

17. `orElseThrow()`

Extremely common in backend applications.

```
Employee employee =
        employeeRepository
                .findById(id)
                .orElseThrow(
                    () -> new RuntimeException(
                        "Employee not found"
                    )
                );
```

Later with Spring Boot, this pattern becomes very common.

---

18. Optional `map()`

Optional can also be transformed.

```
String name =
        employeeOptional
                .map(Employee::name)
                .orElse("Unknown");
```

Flow:

```
Optional<Employee>
        ↓
map(Employee::name)
        ↓
Optional<String>
```

```
        ↓
orElse()
        ↓
String
```

---

19. Optional Best Practice

Don't use `Optional` everywhere.

Avoid unnecessarily doing:

```
class Employee {

    Optional<String> name;
    Optional<String> email;
}
```

Prefer Optional mainly for **return values where absence is meaningful**.

A common example:

```
Optional<Employee> findById(long id);
```

---

20. Real-World Stream Example

Find successful transaction totals per user:

```
Map<String, Double> result =
        transactions.stream()
                .filter(
                    transaction ->
                        "SUCCESS".equals(
                            transaction.status()
                        )
                )
                .collect(
                    Collectors.groupingBy(
                        Transaction::userId,
                        Collectors.summingDouble(
                            Transaction::amount
                        )
                    )
```

```
                );
```

Flow:

```
Transactions
      ↓
filter SUCCESS
      ↓
group by user
      ↓
sum amount
      ↓
Map<UserId, Total>
```

This is the type of Stream pipeline commonly encountered in real backend code.

---

21. Stream Best Practices

Don't modify external state

Avoid:

```
List<String> names =
        new ArrayList<>();

employees.stream()
        .forEach(
            employee ->
                    names.add(employee.name())
        );
```

Prefer:

```
List<String> names =
        employees.stream()
                .map(Employee::name)
                .toList();
```

Don't use streams for everything

A normal loop can sometimes be clearer:

```
for (Employee employee : employees) {

    if (employee.age() >= 30) {
        // logic
    }
}
```

Streams should improve readability, not reduce it.

---

22. Core Stream Mental Model

Remember this pipeline:

```
Collection
    ↓
stream()
    ↓
filter()
    ↓
map()
    ↓
flatMap()
    ↓
collect()
    ↓
Result
```

Not every pipeline needs every operation.

---

23. Level 5 Cheat Sheet

Functional Interfaces

```
Predicate<T>
T → boolean
test()

Function<T, R>
T → R
apply()

Consumer<T>
T → void
```

```
T → void
accept()

Supplier<T>
() → T
get()
```

Streams

```
filter()
→ select elements

map()
→ transform elements

flatMap()
→ flatten nested elements

reduce()
→ combine into one value

collect()
→ gather/process results

groupingBy()
→ group into multiple categories

partitioningBy()
→ split into true/false

counting()
→ count

summingDouble()
→ sum values
```

Optional

```
Optional<T>
    ↓
map()
    ↓
orElse()
orElseThrow()
```

24. Most Important Patterns

Filtering

```
employees.stream()
        .filter(e -> e.age() >= 30)
        .toList();
```

Transformation

```
employees.stream()
        .map(Employee::name)
        .toList();
```

Filter + Transform

```
employees.stream()
        .filter(e -> e.age() >= 30)
        .map(Employee::name)
        .toList();
```

Flattening

```
employees.stream()
        .flatMap(
            e -> e.projects().stream()
        )
        .toList();
```

Grouping

```
employees.stream()
        .collect(
            Collectors.groupingBy(
                Employee::department
            )
        );
```

Group + Count

```
employees.stream()
        .collect(
            Collectors.groupingBy(
                Employee::department,
                Collectors.counting()
            )
        );
```

Group + Sum

```
employees.stream()
        .collect(
            Collectors.groupingBy(
                Employee::department,
                Collectors.summingDouble(
                    Employee::salary
                )
            )
        );
```

Optional

```
repository.findById(id)
        .orElseThrow(
            () -> new RuntimeException(
                "Not found"
            )
        );
```

---

25. Level 5 Final Checklist

```
✅ Lambda Expressions
✅ Functional Interfaces

✅ Predicate
✅ Function
✅ Consumer
✅ Supplier

✅ Method References

✅ Stream Pipeline
✅ filter()
✅ map()
```

```
✅ flatMap()
✅ reduce()
✅ collect()

✅ groupingBy()
✅ counting()
✅ summingDouble()
✅ partitioningBy()

✅ Optional
✅ Optional.map()
✅ orElse()
✅ orElseThrow()

✅ Stream best practices
```

Level 5 Goal

You don't need to memorize the entire Java Stream API.

You should be able to read and write code like:

```
transactions.stream()
        .filter(t -> "SUCCESS".equals(t.status()))
        .map(Transaction::amount)
        .toList();
```

and:

```
employees.stream()
        .collect(
            Collectors.groupingBy(
                Employee::department,
                Collectors.counting()
            )
        );
```